

Take-Home Challenge

Build a Projects API in Clojure

This exercise mirrors our work: shape an underspecified request, build an API in Clojure. Use AI as you normally would; document how you verified its output. We will score with the rubric at the end of the document.

- File-based DB: SQLite or DuckDB
- No external services required

Scenario

Product needs a small **Projects** service. Required endpoints

- `GET /projects` — list projects (you decide pagination/sorting; document it).
- `POST /projects` — create a project.
- `GET /projects/{id}` — fetch a single project.

Minimal domain model

A **Project** must include:

- `id` — unique identifier (you pick the type; justify it)
- `name` — human-readable name
- `status` — at least one terminal/archival value (define the enum and default)
- `created_at` — server timestamp

You may add fields if you explain why.

API versioning

- Implement and document an API versioning approach. Justify your choice

Contracts & validation

- Provide an OpenAPI 3.1.1 document describing the API's final behavior.
- Use Malli schemas for:
 - request validation/coercion
 - response validation (at least for the primary success responses)

Be explicit about:

- validation rules for `name`
- what error payloads look like
- status codes for invalid input, not found, and conflicts

Stateful system design

We care about production-grade service shape even for a small challenge. Your implementation must include and document:

- how you manage stateful components (HTTP server, DB connection pool, config)
- lifecycle (start/stop) and local dev ergonomics
- how you do DB connection pooling (or why you intentionally didn't)

You can pick any component system / approach, but explain tradeoffs.

Data storage

- Use SQLite or DuckDB for local development (or another you prefer, but explain reasoning)
- Include a simple migration (SQL file or a tool of your choice) that creates the table(s).
- Seed data to make `GET /projects` useful.

Note: For this take-home, manage the file DB via your app/migrations, not Terraform.

Terraform deliverable (code-only; no cloud account)

Create a small Terraform module (in `/terraform`) that *would* provision, in production:

- a Snowflake database + schema
- a `projects` table that matches your model

Include variables for names/roles and a short module README. Do **not** apply; code should at least `terraform validate`.

Tests

- Write tests
- Cover happy-path and at least one edge case you identified.

Deliberate ambiguities you must resolve

- Pagination shape (limit/offset or cursor) and default page size.
- Validation for `name` (length, charset, uniqueness semantics).
- `status` values and default on create.
- Error payloads and status codes (e.g., 400 vs 409 vs 422).
- ID strategy (UUID vs integer vs other) and why it fits a file DB.

Document choices in `DESIGN.md` and reflect them in tests and OpenAPI.

What to submit

1. A PR to `main` in the Clojure repository with:
 - A Clojure implementation from scratch with Malli schemas.
 - The OpenAPI spec.
 - `migrations/` for your DB schema.
 - `/terraform` module
2. `DESIGN.md` including:
 - Your acceptance criteria and assumptions.
 - Rationale for library choices.
 - Notable AI prompts (optional) and how you verified AI output.
 - Commands to run each API locally and run tests.

Optional extensions

Pick one if you have time:

- Filtering in `GET /projects` (e.g., by `status`).
- A `milestones` sub-resource:
 - `POST /projects/{id}/milestones`
 - `GET /projects/{id}/milestones`

DESIGN_TEMPLATE.md

1. Problem definition

- Final acceptance criteria (functional + non-functional)
- Out-of-scope items
- Open questions (if any)

2. Data model

- Tables and fields (+ rationale)
- ID strategy and trade-offs for file DBs (SQLite vs DuckDB)

3. API contract

- Summary of endpoints
- Error model
- Link to OpenAPI (or embed)
- Versioning

4. Validation

- Rules for inputs (name, status, etc.)
- How validation is enforced

5. Implementation notes

- Library choices and why
- Project layout
- Testing approach
- Performance or security considerations
- Stateful component management
- DB pooling

6. Terraform module (code-only)

- What resources this would create in Snowflake
- Variables, outputs, and review surfaces

7. AI usage (optional but encouraged)

- Prompts that materially influenced the solution
- How you verified/modified AI output

8. Run & test

- Commands to start each service and run tests
- Any environment variables or config required

Scoring Rubric

Scoring Rubric (100 points)

Category	4 – Exceptional	3 – Strong	2 – Adequate	1 – Concern	Weight
Spec & Acceptance Criteria	Comprehensive spec; clear out-of-scope; thoughtful questions surfaced early.	Main requirements captured; minor gaps.	Basic list; notable gaps.	Little/no spec beyond README.	15% ▾
Solution Correctness	All scenarios pass; robust error handling; no critical bugs.	Happy-path + common edges pass.	Happy-path passes; edges missing.	Happy-path fails.	25% ▾
Code Quality	Readable, idiomatic, modular; good tests; clear structure.	Mostly idiomatic; tests cover main flows.	Mixed styles; limited tests.	Confusing or fragile; little/no tests.	25% ▾
Company Practices	OpenAPI present; Malli used in Clojure; migrations included; sane logging/structure.	Minor deviations, easy to fix.	Several guideline breaches.	Major violations (secrets, no OpenAPI/Malli).	15% ▾
AI-Assisted Judgment	Transparent prompt logs; verifies & improves AI output.	Uses AI with verification tests.	Heavy reliance; minimal checks.	Paste-and-pray; misses obvious flaws.	20% ▾

Passing: ≥ 70 points overall and no 1-scores in Correctness, Code Quality, or AI-Judgment.